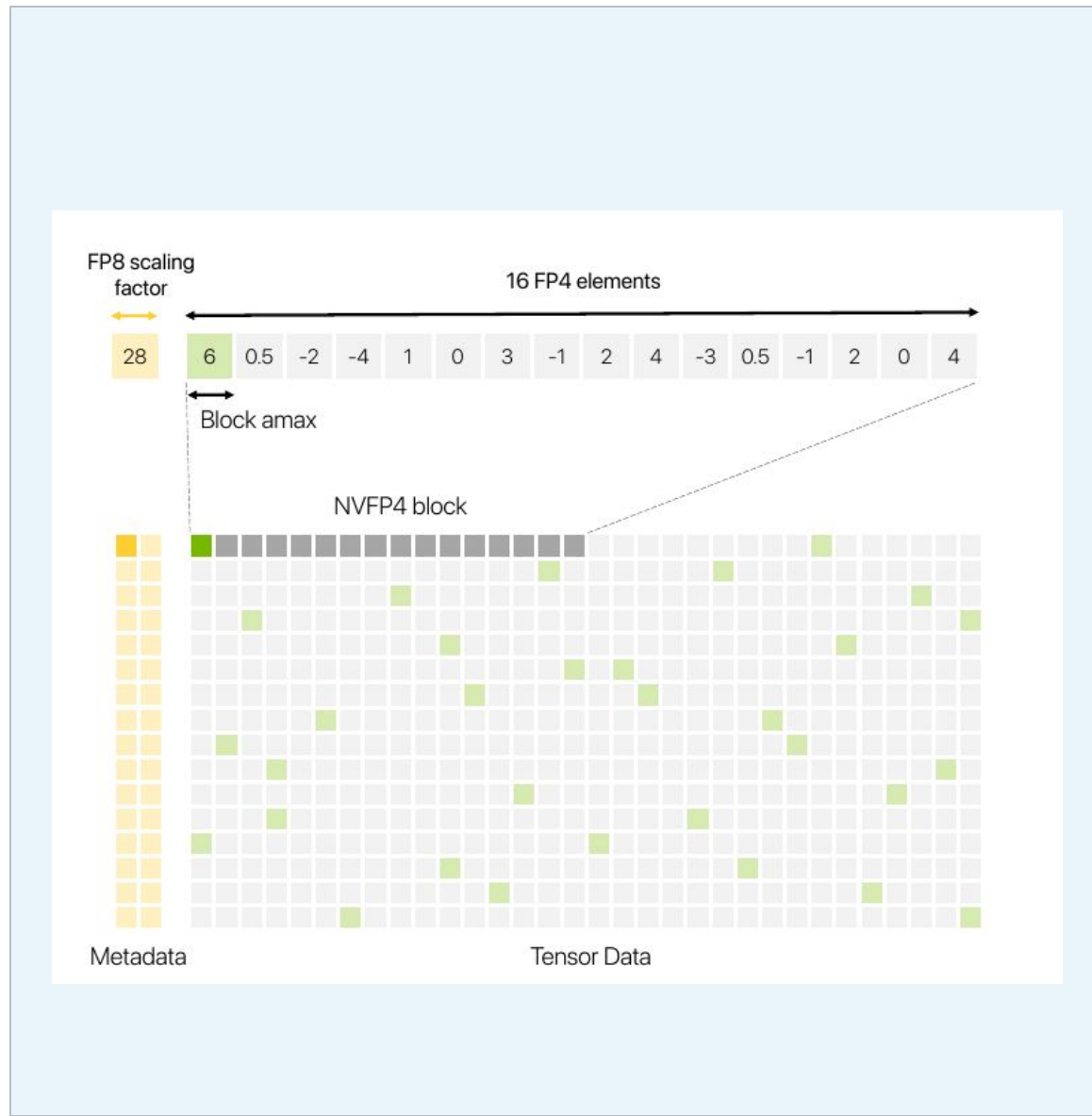


Pretraining Large Language Models with NVFP4

4비트(NVFP4)로 12B 모델을 10T 토큰까지
안정적으로 학습한 방법



논문의 핵심 주장은 한 문장으로 요약됨

NVFP4 자체만으로는 부족하다.

Sensitive Layer는 BF16/FP32 + RHT + 2D weight scaling + gradient stochastic rounding을 함께 적용 필요

12B 모델이 10T 토큰 동안 FP8과 비슷하게 학습된다.

RHT (Random Hadamard Transform) : [0.2, 0.1, 0.3, 0.2, 15.0, 0.1, 0.2, 0.3] -> 평탄화 (이상치 15.0가 있다)

Gradient Stochastic Rounding : 1.0 $\leftarrow$ 35% 1.65(원래값) 2.0 $\leftarrow$ 65%

중요: ‘모든 연산이 FP4’라는 논문이 아니라, linear GEMM 입력을 선택적으로 NVFP4로 내린 mixed-precision 학습 논문입니다.

NVFP4 특성부터 시작해 실험 기반으로 최적 훈련 방법 도출

01 ● NVFP4 특성

FP4, block scale, two-level scaling

02 ● 12B / 10T 실험

무엇을 양자화했고 결과가 어땠는가

03 ● Training Methodology

naive FP4가 실패하는 이유와 네 가지 수정

04 ● Ablation과 한계

어떤 장치가 실제로 필요했는가

FP4의 매력은 GEMM 처리량과 메모리 절감

2-3×

FP8 대비 Blackwell FP4
수학 처리량(칩에 따라)

≈ 1/2

FP8 operand 대비
메모리 사용량

10T

한 번의 pretraining에서
누적되는 token 규모

하지만 4 bit는 오차 여유가 거의 없습니다

값의 범위가 좁고 표현 가능한 숫자가 적어서 outlier, rounding bias, forward/backward 불일치가 곧바로 학습 불안정으로 이어집니다.

양자화는 연속적인 값을 제한된 칸에 넣는 일입니다

원래 값

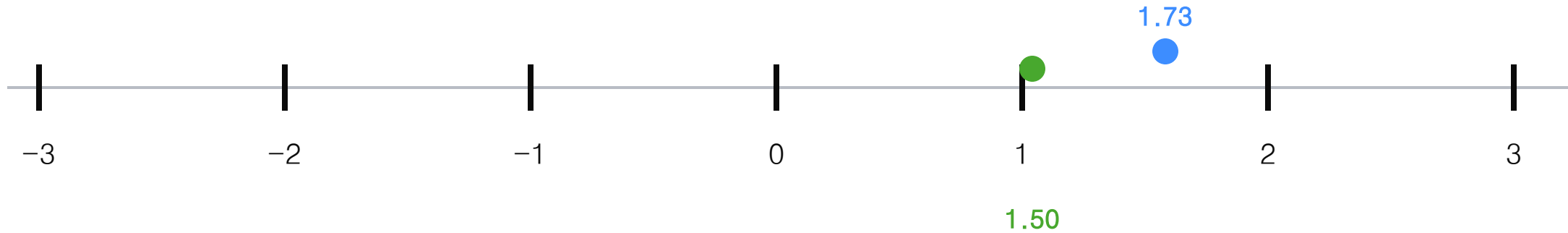
1.73

scale로 나누고
가까운 표현값 선택

복원 값

1.50

오차 -0.23



scale은 칸의 간격을 정하고, rounding은 어느 칸으로 보낼지 정합니다.

비트가 줄수록 표현 범위와 정밀도를 상실

BF16

1 sign · 8 exponent · 7
mantissa

넓은 범위
상대적으로 촘촘한 표현

FP8 E4M3

1 sign · 4 exponent · 3
mantissa

학습에 널리 쓰이는
8-bit baseline

FP4 E2M1 (NVFP4)

1 sign · 2 exponent · 1
mantissa

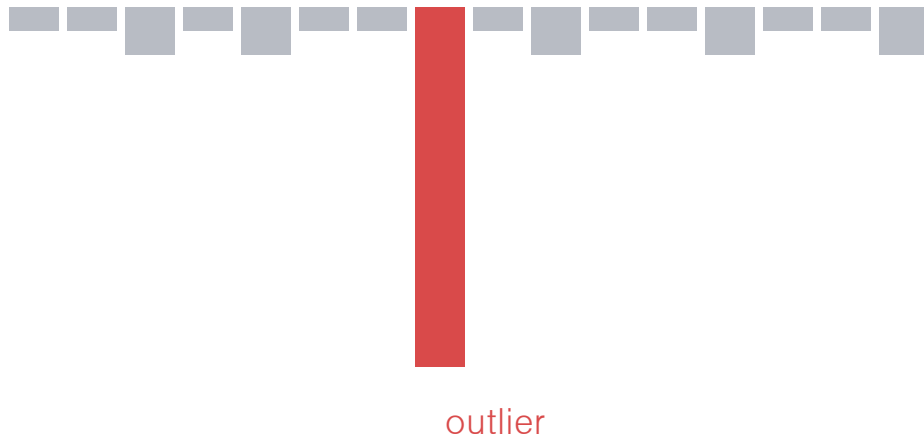
표현값이 매우 적어
scale 없이는
실용적이지 않음

NVFP4의 핵심은 FP4 element에 더 정교한 scale metadata를 붙이는 것입니다.

한 개 scale보다 작은 block별 scale이 오차를 줄임

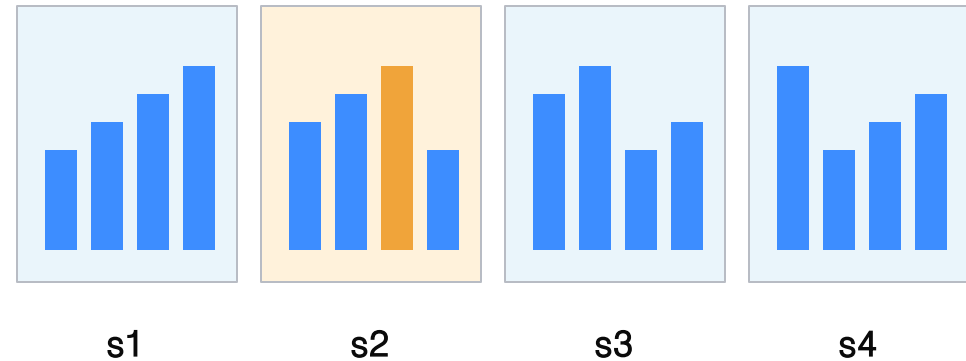
Tensor 전체에 scale 1개

큰 outlier 하나가 전체 간격을 벌림



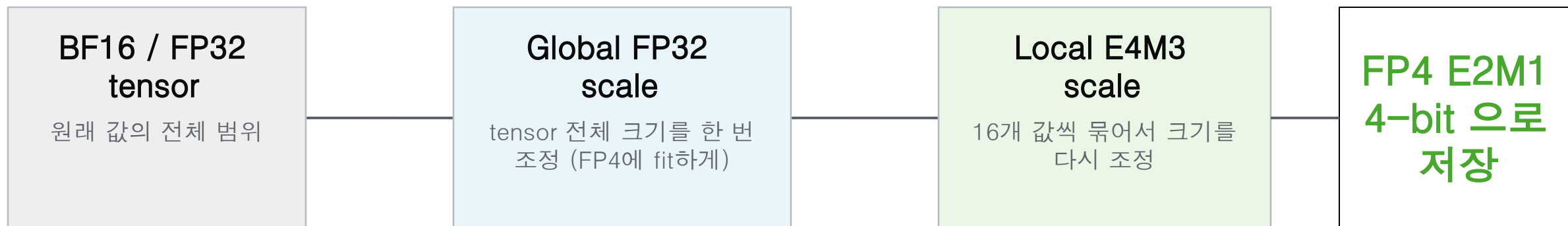
16개마다 scale 1개

각 block이 자기 범위에 맞춰 확대·축소



Microscaling = data 옆에 작은 scale metadata를 저장해 local dynamic range를 맞추는 방식

NVFP4는 tensor scale과 block scale을 두 단계로 사용



혼동 주의

two-level scaling은 'FP32 tensor scale + E4M3 block scale'입니다. 뒤의 2D scaling(16×16 weight tile)과는 다른 개념입니다.

NVFP4는 MXFP4보다 block이 작고 scale이 정교합니다

항목	NVFP4	MXFP4	학습에 미치는 의미
Element	E2M1	E2M1	둘 다 4-bit data
Block size	16	32	작을수록 local range에 잘 맞춤
Block scale	E4M3	UE8M0	NVFP4 scale이 더 촘촘함
두 단계 scale	FP32 + E4M3	power-of-two 중심	작은 값의 zero화와 outlier 오차 감소

숫자 형식 차이가 token 효율 차이로 이어집니다

같은 training recipe를 적용한 8B 실험에서 MXFP4는 NVFP4 loss를 맞추기 위해 36% 더 많은 token이 필요했습니다.

power of two 설명

$$e = -3 \rightarrow 2^{-3} = 0.125$$

$$e = -2 \rightarrow 2^{-2} = 0.25$$

$$e = -1 \rightarrow 2^{-1} = 0.5$$

$$e = 0 \rightarrow 2^0 = 1$$

주요 실험은 12B hybrid model을 10T token까지 학습

12B

Nemotron-H 계열
hybrid
Mamba-Transformer

10T

pretraining tokens

8K

sequence length

WSD

80% stable +
20% decay

구조

Mamba-2 + Self-Attention + FFN block 혼합

비교

동일 architecture의 FP8 baseline

평가

학습 loss + knowledge / math / code / commonsense task

12B run에서 NVFP4는 linear GEMM 입력에만 적용

NVFP4

Linear layer의 세 Fprop, Dgrad, Wgrad GEMM operand를 NVFP4로 양자화

GEMM output은 BF16 또는 FP32

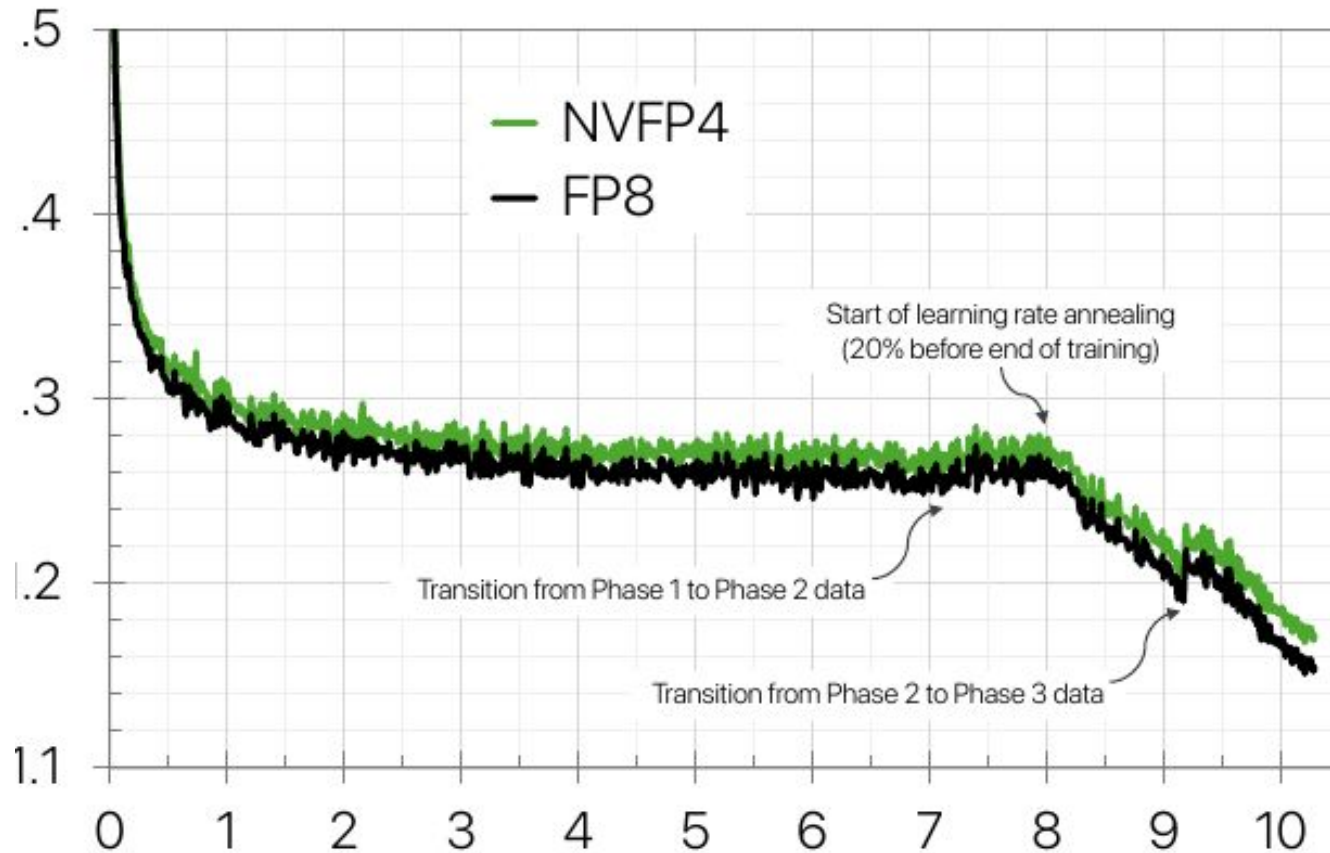
GEMM의 dot-product 합산결과를 FP4보다 높은 정밀도로 누적수행

BF16 / FP32

- embedding / output head / norm / nonlinearity
- attention softmax, QK, score-V GEMM
- FP32 master weights / accumulated Wgrad / optimizer
- BF16 tensor-parallel reduction
- first 2 + last 8 blocks의 linear layers

따라서 'full-stack FP4 training'이라고 부르면 과장.

10T token 동안 NVFP4 loss는 FP8을 가깝게 따라갑니다



Stable phase

relative loss gap < 1%

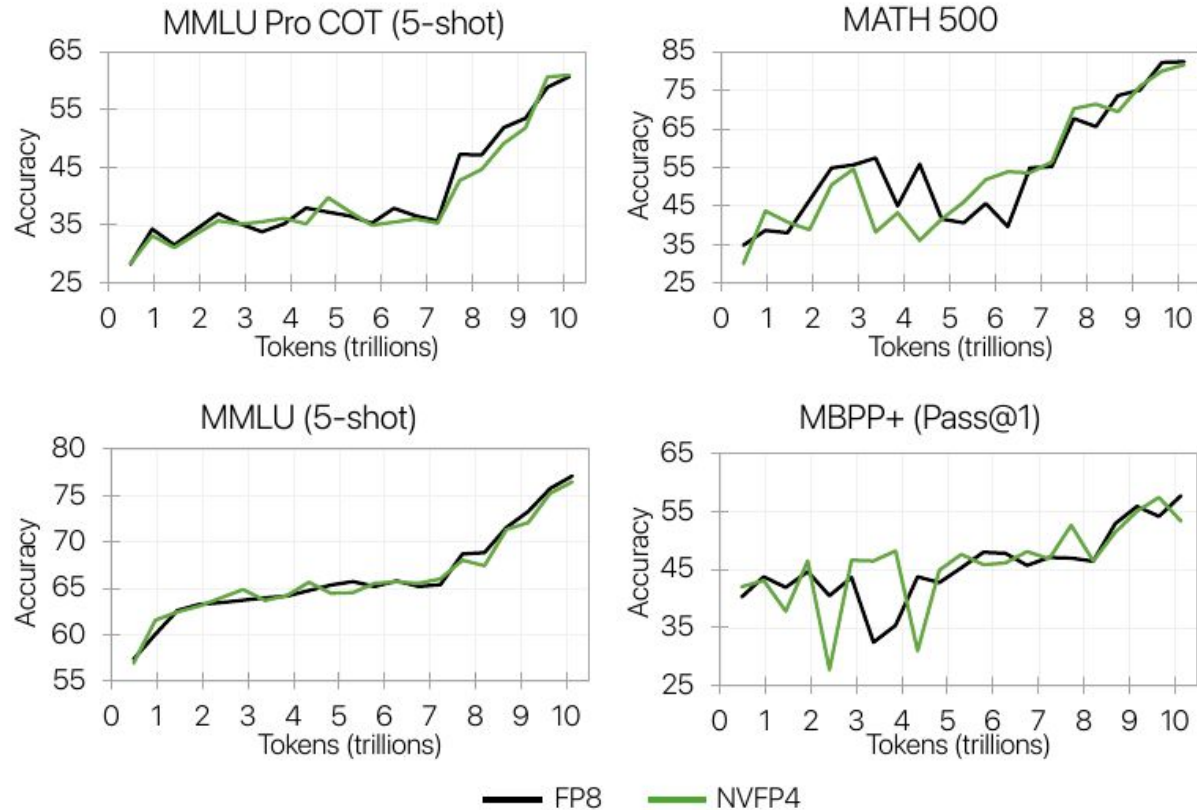
Decay phase

끝에서 약 1.5%+로 확대

8T: LR decay 시작

9T: dataset blend 변경

대부분의 downstream accuracy는 FP8과 비슷합니다



Task	FP8	NVFP4
MMLU-Pro	62.62	62.58
GSM8K CoT	89.08	92.27
MATH	83.32	81.48
HumanEval+	59.93	57.43
MBPP+	59.11	55.91

Headline 예: MMLU-Pro 62.58 vs 62.62

결과를 읽을 때 세 가지 조건을 붙여야 합니다

- | | | |
|----|--------------|--|
| 01 | 평가는 BF16 | Table 2의 task accuracy는 NVFP4 inference가 아니라 BF16 evaluation입니다. |
| 02 | loss gap은 남음 | stable phase <1%, decay 끝에서는 약 1.5%+로 벌어집니다. |
| 03 | code는 약세 | HumanEval+와 MBPP+에서 FP8보다 낮고, 저자는 noisy checkpoint 가능성을 제시합니다. |

Naive NVFP4는 ‘전부 FP4 + 기본 scale + RNE로 발산

Appendix E의 base method

- 모든 linear layer를 NVFP4
- 모든 tensor에 1×16 scaling
- 모든 tensor에 round-to-nearest-even (RNE)
- 추가 안정화 장치 없음

조기 발산

FP4가 나빠서가 아니라
오차가 들어가는 위치마다
다른 처방이 필요

Section 4는 이 실패 원인을 하나씩 제거한 이야기입니다.

논문의 recipe는 네 가지 실패에 네 가지 솔루션을 실험

1

민감 layer의 range 부족

일부 linear layer를 BF16/MXFP8로 유지

2

Wgrad input의 block outlier

16×16 Random Hadamard transform

3

transpose 뒤 weight 표현 불일치

weight에 16×16 2D scaling

4

gradient rounding bias

gradient에 stochastic rounding

한 linear layer에는 세 개의 다른 역할의 GEMM이 등장

Fprop

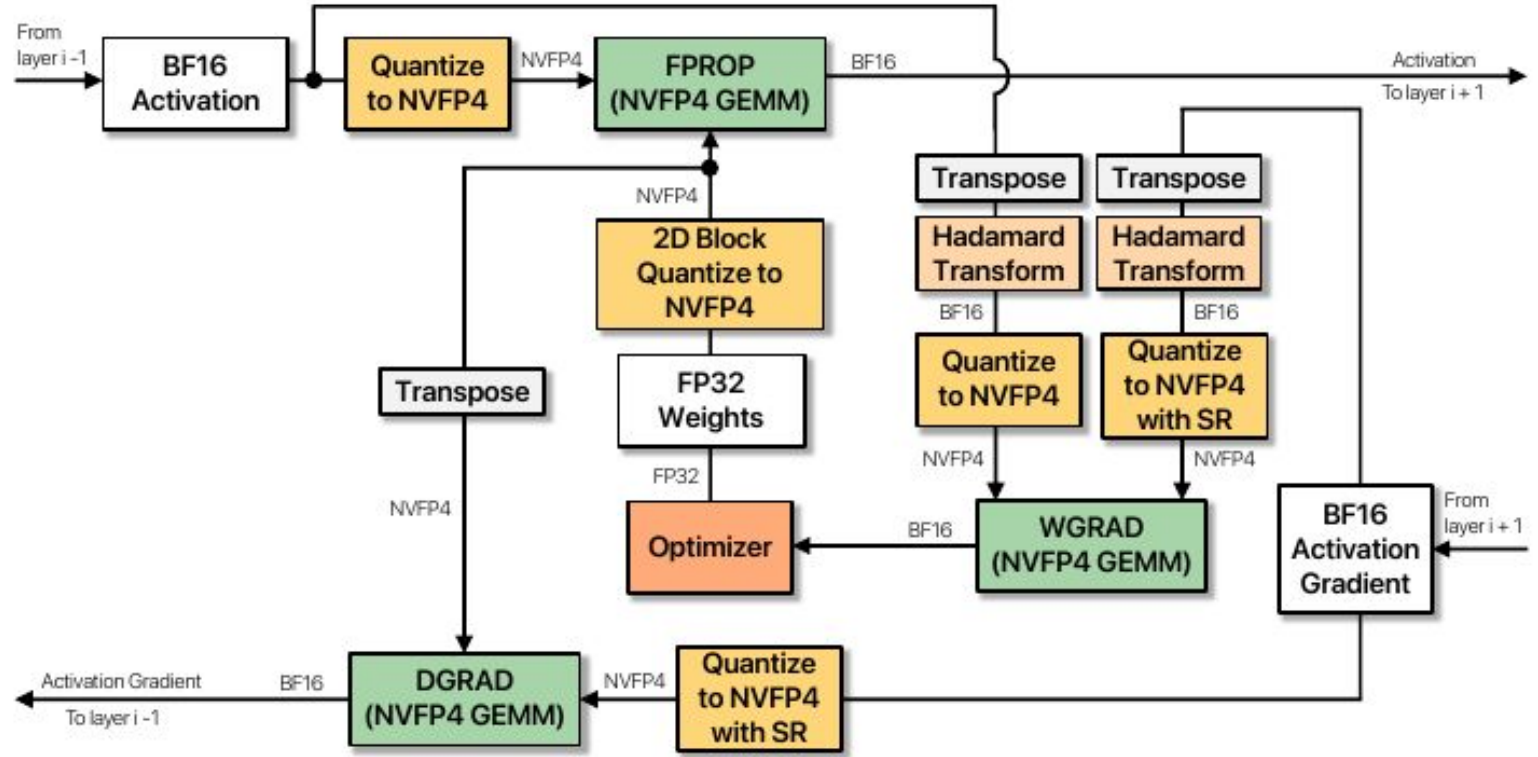
activation $\times$ weight
→ 다음 layer activation

Dgrad

activation gradient 계산
→ 이전 layer로 전달

Wgrad

weight gradient 계산
→ optimizer가 weight 갱신



같은 FP4 처방을 세 GEMM에 무조건 적용하면 안 됩니다.

Fix 1 · 마지막 쪽 민감 layer를 고정밀로 남깁니다

왜?

마지막 linear layer들은 Wgrad quantization error가 크고, FP4보다 더 넓은 range-mantissa를 요구합니다.



16%

12B main run에서 BF16로
남긴 linear layers

Ablation은 last 4 blocks만 BF16이어도 안정적인 수 있음을
보였습니다.

Fix 2 · RHT는 큰 outlier 하나를 여러 값으로 분산



Orthogonal transform이라 두 operand에 맞춰 적용하면 dot product 자체는 보존할 수 있습니다.

RHT는 Wgrad 입력에만 적용하는 것이 가장 유리

적용 위치	결과	논문 해석
Wgrad inputs	개선	outlier 완화 이득이 큼
Fprop inputs	악화	추가 quantization error가 이득 상쇄
Dgrad inputs	악화	추가 quantization error가 이득 상쇄

선택한 설정

Wgrad input에 16×16 RHT
한 개 fixed random sign vector를 전체
학습에서 공유

왜 weight에는 안 쓰나?

weight를 서로 다른 방향으로
변환·양자화하면 forward/backward
표현 불일치가 더 커집니다.

‘outlier 제거는 항상 좋다’가 아니라, 위치별 추가 오차까지 비교해야 합니다.

여기서 Q 를 RHT 행렬이라고 합시다. Hadamard transform은 orthogonal이므로

$$Q^T Q = I$$

입니다.

그러면 dY 와 X 를 똑같은 RHT로 변환해서

$$dY' = QdY$$

$$X' = QX$$

로 만들 수 있습니다.

그런데 계산해 보면

$$dY'^T X' = (QdY)^T (QX)$$

$$= dY^T Q^T QX$$

$$= \boxed{dY^T X}$$

입니다.

즉,

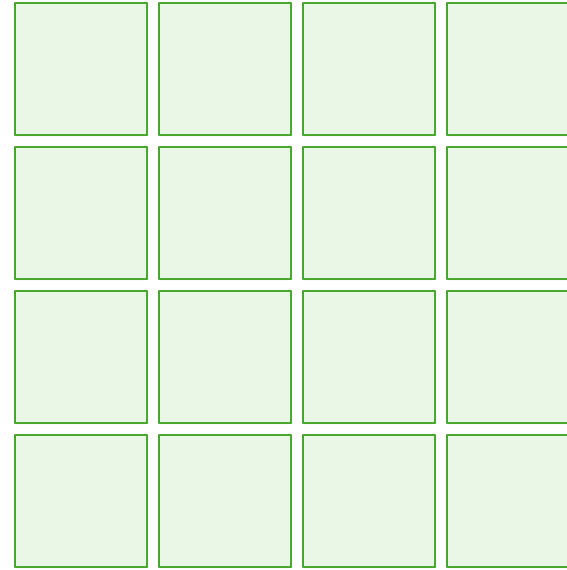
RHT를 해도 Wgrad 결과는 원래와 완전히 동일

16×16 2D scaling은 transpose 전후 동일한 weight scale을 재사용할 수 있습니다

1×16 scaling

- Forward: W^T 방향으로 16개씩 scaling
- DGRAD: W 방향으로 16개씩 scaling
- → transpose하면 block 구성이 달라짐
- → 별도의 scale / quantization
- 필요 $Q(W^T) \neq Q(W)^T$

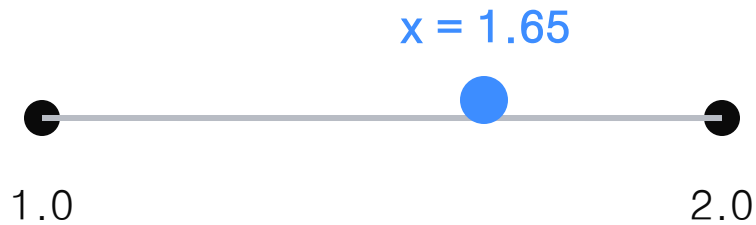
16×16 2D scaling



16×16 tile 단위로 하나의 scale 사용
 transpose해도 동일한 256개 weight가 하나의 tile
 → 같은 scale 재사용 가능
 → quantized weight도 transpose하여 재사용 가능
 $Q(W^T) = Q(W)^T$

stochastic rounding은 작은 gradient의 방향 편향을 줄입니다

표현 가능한 두 값



Stochastic Rounding

1.0으로 35%
2.0으로 65%

여러 update의 평균에서는
원래 값 1.65를 보존

RNE

항상 2.0 → 같은 방향 오차가 반복될
수 있음

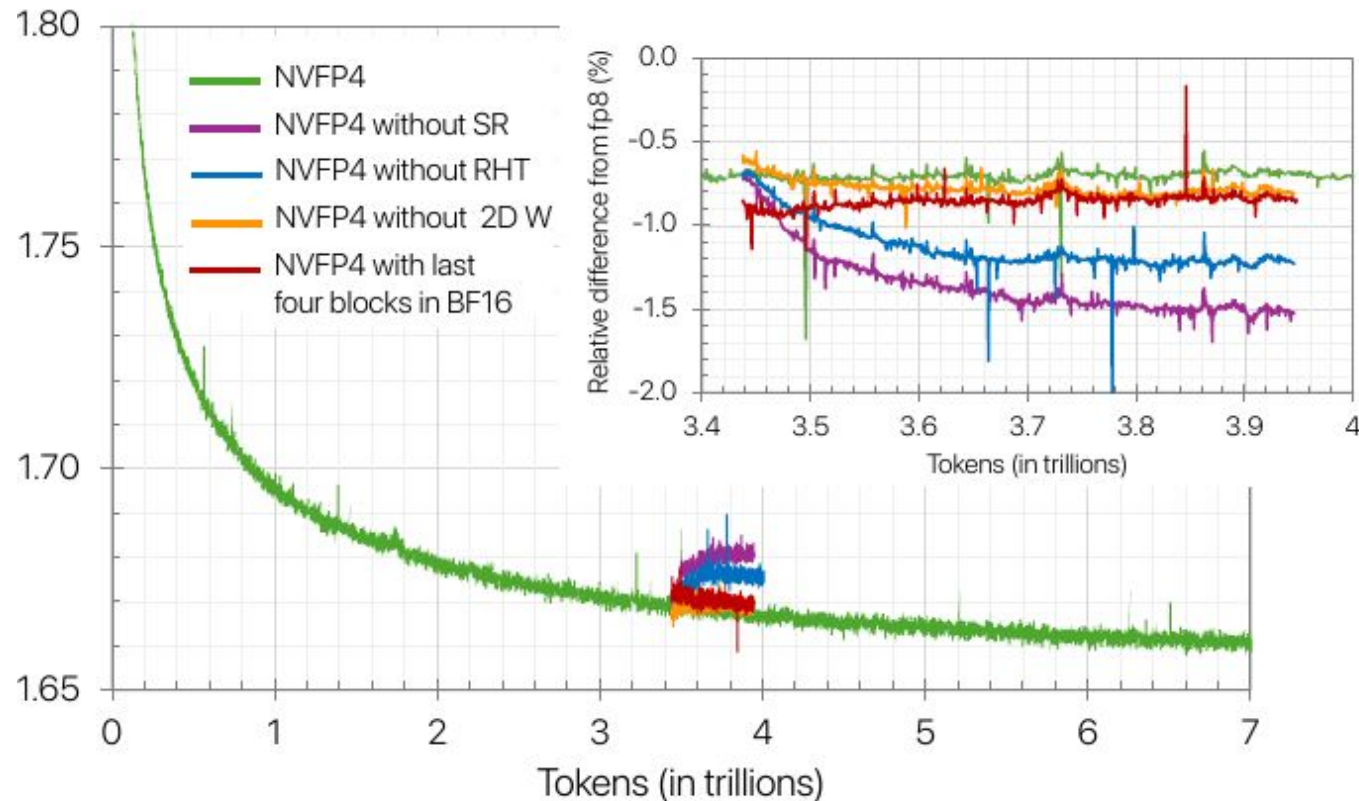
논문 결과: gradient에는 필수, weight·activation에 적용하면 오히려 추가 quantization noise로
악화

정밀도와 rounding은 tensor 역할마다 다르게 배치됩니다

대상	GEMM input / 저장 정밀도	Rounding / scaling	이유
Weight	NVFP4 input; FP32 master	RNE + 16×16 2D	forward/backward consistency
Activation	NVFP4 input; BF16 흐름	RNE + 1×16	finer block로 accuracy
Gradient → Dgrad	NVFP4 input	SR + 1×16	gradient bias 감소
Gradient → Wgrad	RHT 후 NVFP4 input	SR + 1×16	outlier + bias 동시 완화
GEMM output	BF16 / FP32	해당 없음	accumulation 안정성

짧게 기억하기: W/A는 RNE, gradient는 SR, weight만 2D, Wgrad input만 RHT

하나씩 제거하면 12B training loss가 모두 나빠집니다



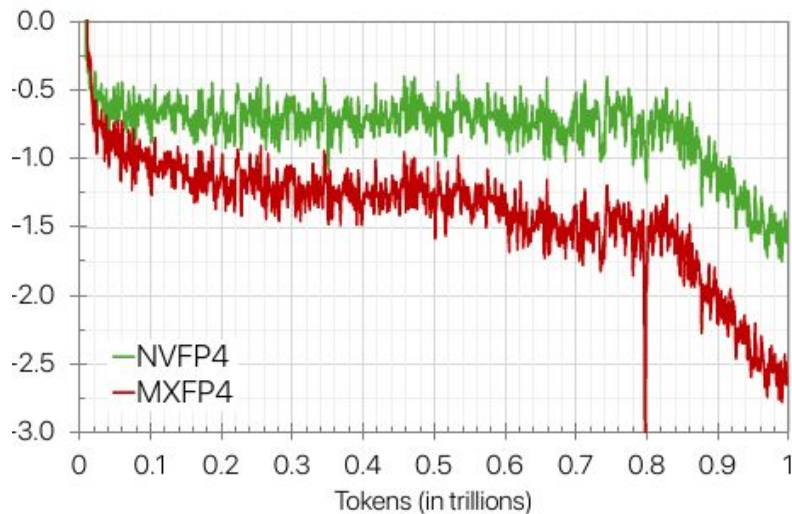
제거한 장치

- stochastic rounding
- Wgrad RHT
- 2D weight scaling
- BF16 layer 수 축소

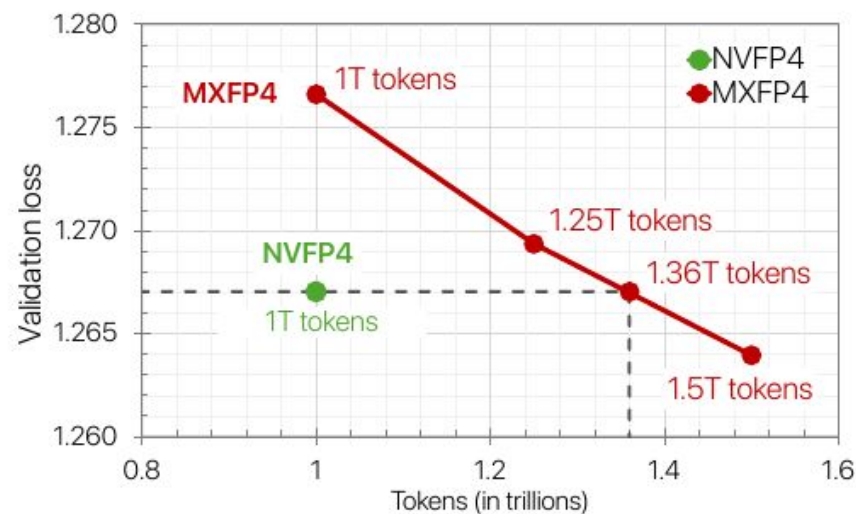
해석

각 장치는 서로 다른 오차를 막고, 10T horizon에서는 결합이 필요합니다.

같은 recipe에서도 NVFP4가 MXFP4보다 token-efficient합니다



Relative difference between training loss of BF16 (baseline) and NVFP4 and MXFP4 pretraining



(b) Final validation loss for NVFP4 and MXFP4 pretraining with different number of tokens

1.5%

NVFP4 relative loss error

2.5%

MXFP4 relative loss error

+36%

MXFP4가 같은 loss에 도달한 추가 token

8B model · 1T-token 기준 비교

이 논문은 가능성을 보였지만 full-stack 효율은 아직 증명하지 않았습니다

입증한 것

- 12B / 10T 장기 NVFP4 안정 학습
- FP8에 가까운 loss·task accuracy
- 네 장치의 ablation 근거
- NVFP4 > MXFP4 token efficiency

남은 질문

- end-to-end throughput / energy
- attention·optimizer·communication FP4
- 더 큰 model·MoE로의 확장
- high-precision layer 자동 선택

정확한 표현: 'FP4 pretraining의 실용적 경로'이지 '모든 학습을 FP4로 끝냈다'가 아닙니다.

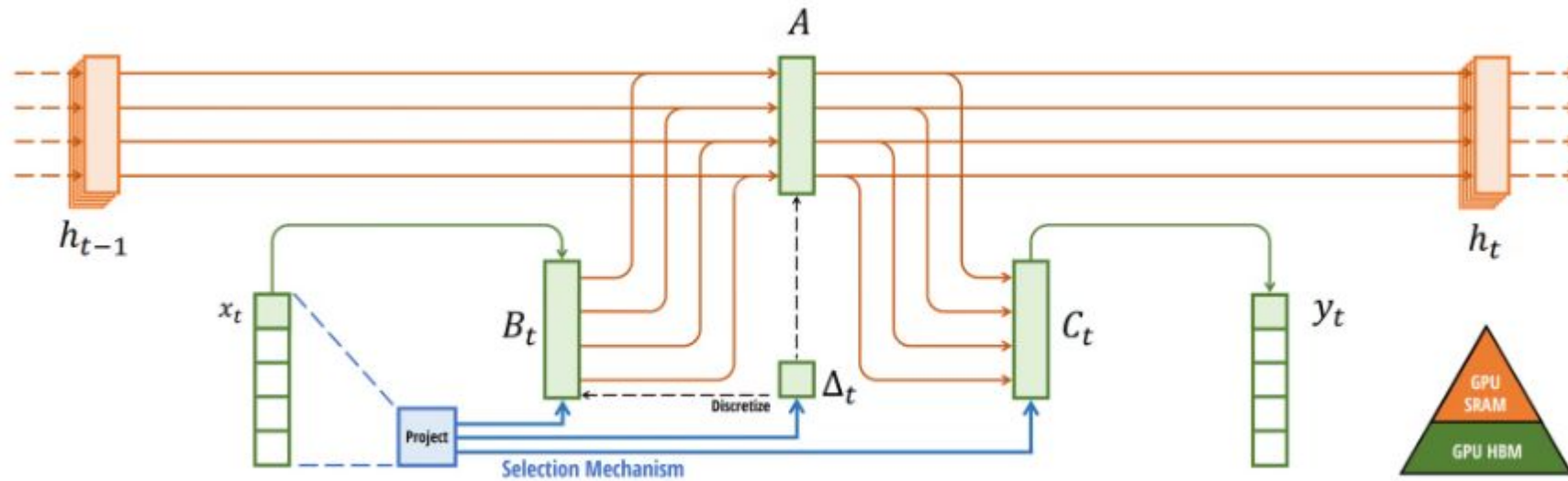
FP4 training은 '낮은 bit'보다 '오차의 위치'를 설계하는 문제입니다

FORMAT

16-value block + E4M3 scale + FP32 tensor scale

ST/

EVII



만들었습니다.

부록 · 마지막 layer를 보호해야 했던 실험적 이유

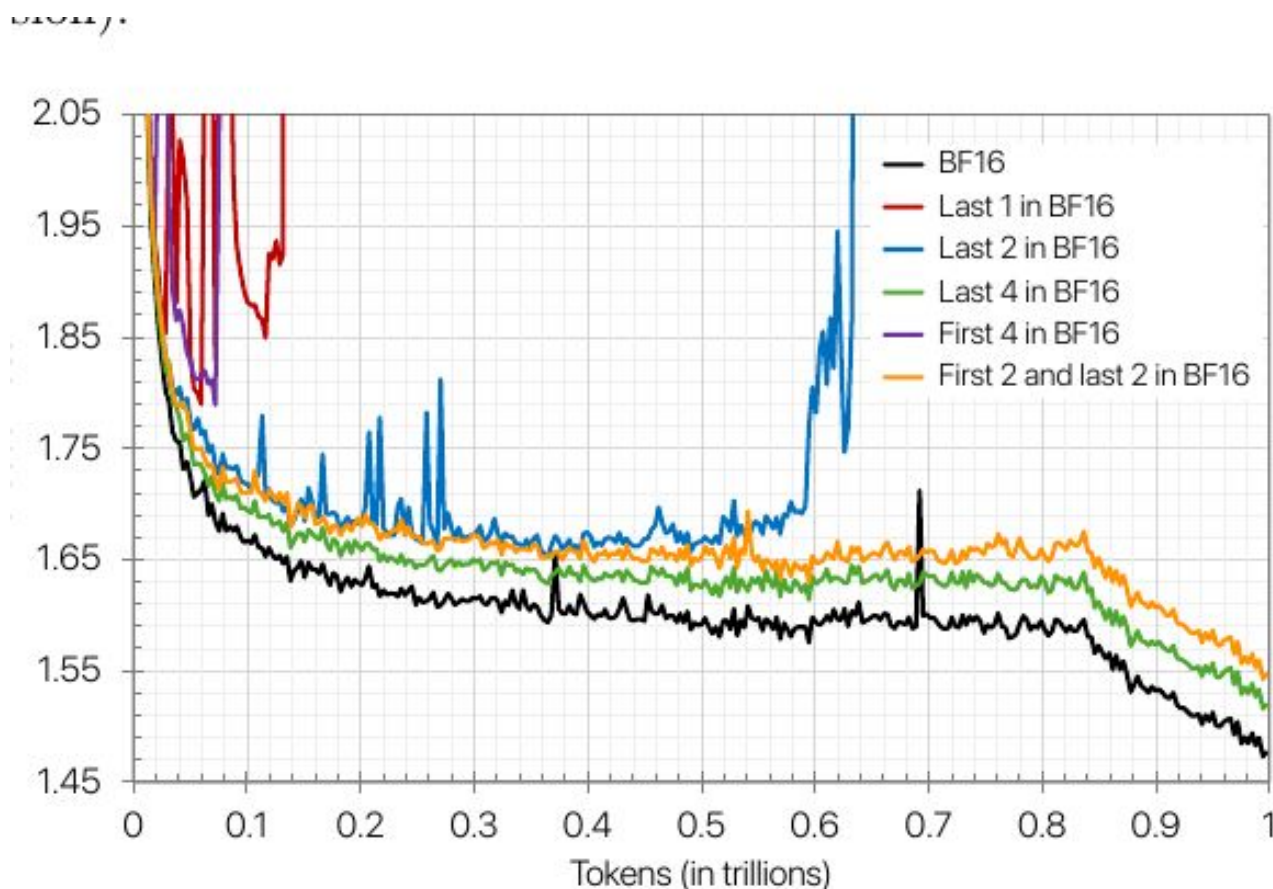
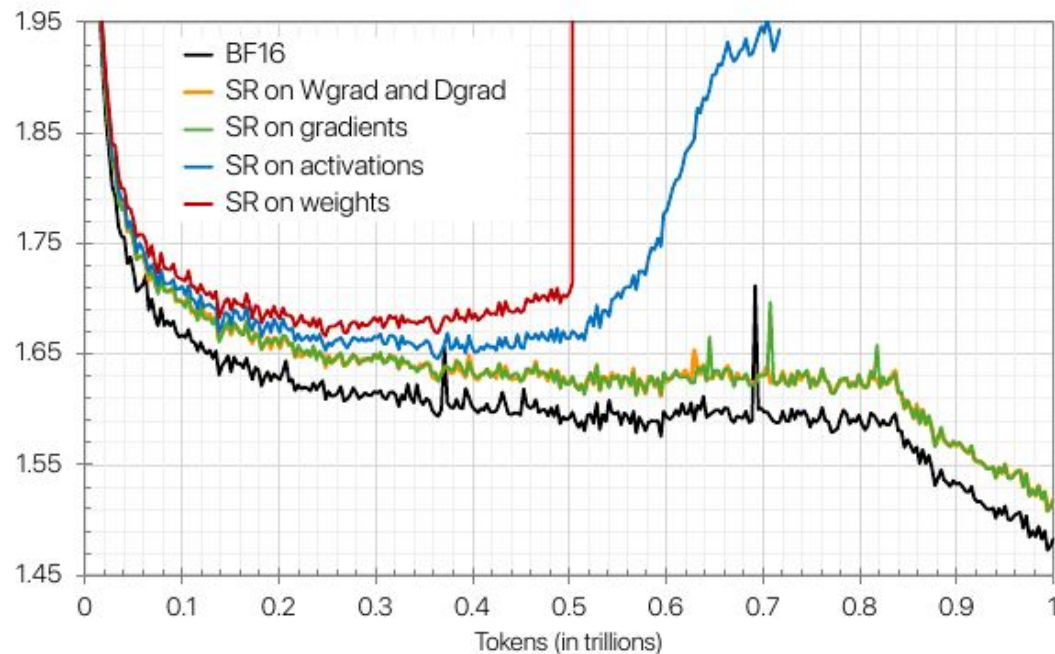


Figure 9

- last 1 / last 2만 BF16: 불안정
- last 4 BF16: 안정적 수렴
- first 4만 BF16: 불안정
- first 2 + last 2: 안정 가능

핵심은 ‘몇 개’뿐 아니라
‘어디의 layer인가’
입니다.

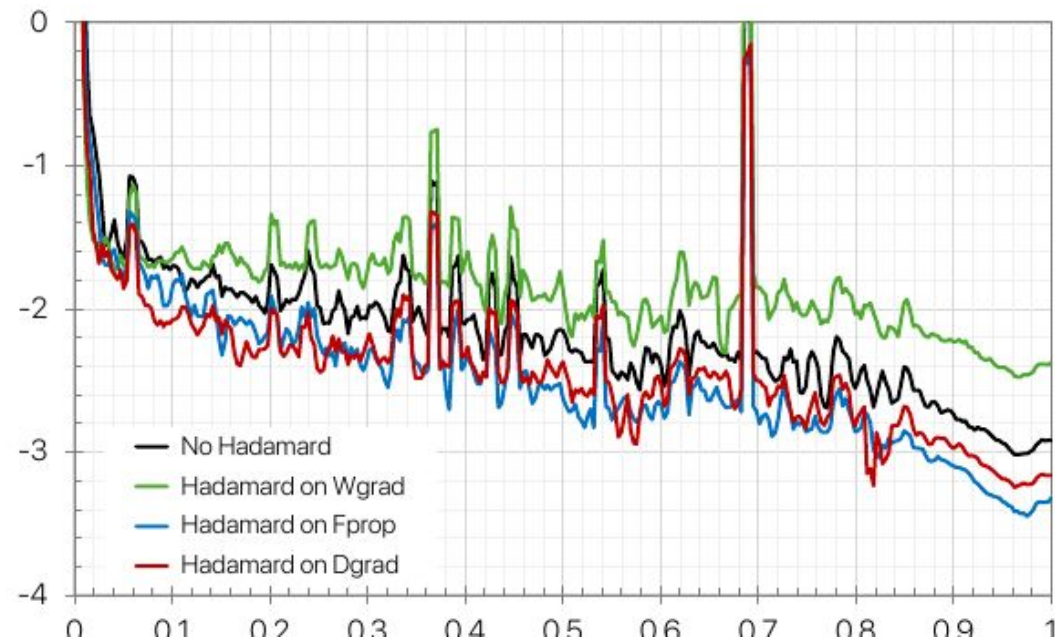
부록 · SR과 RHT는 적용 위치가 성패를 가릅니다



SR

gradient는 안정 · weight/activation은 발산

hard transforms

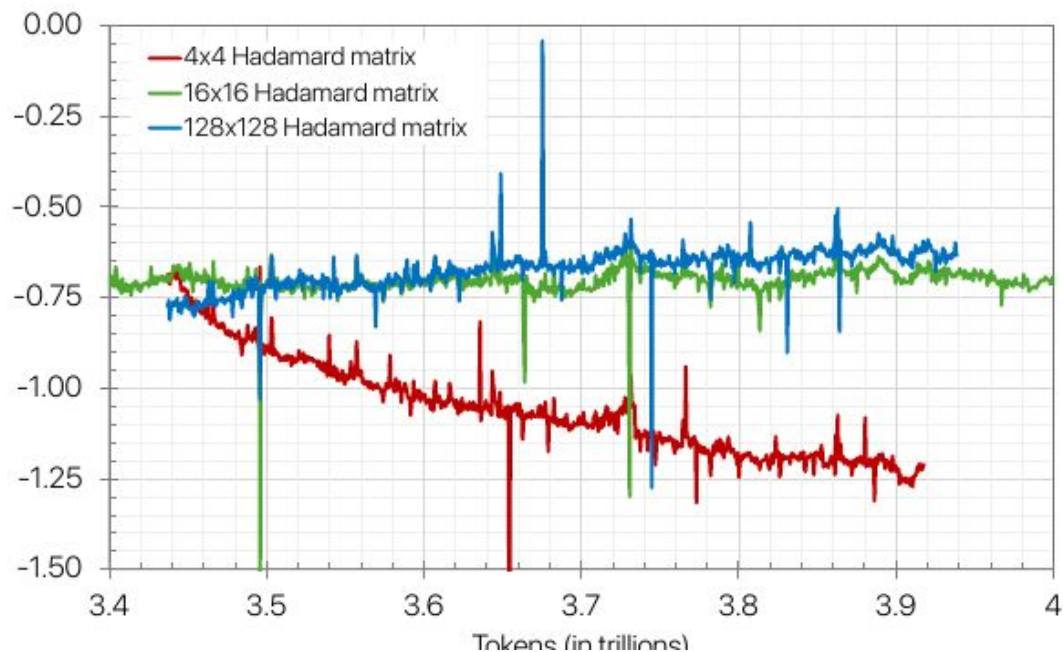


RHT

Wgrad는 개선 · Fprop/Dgrad는 악화

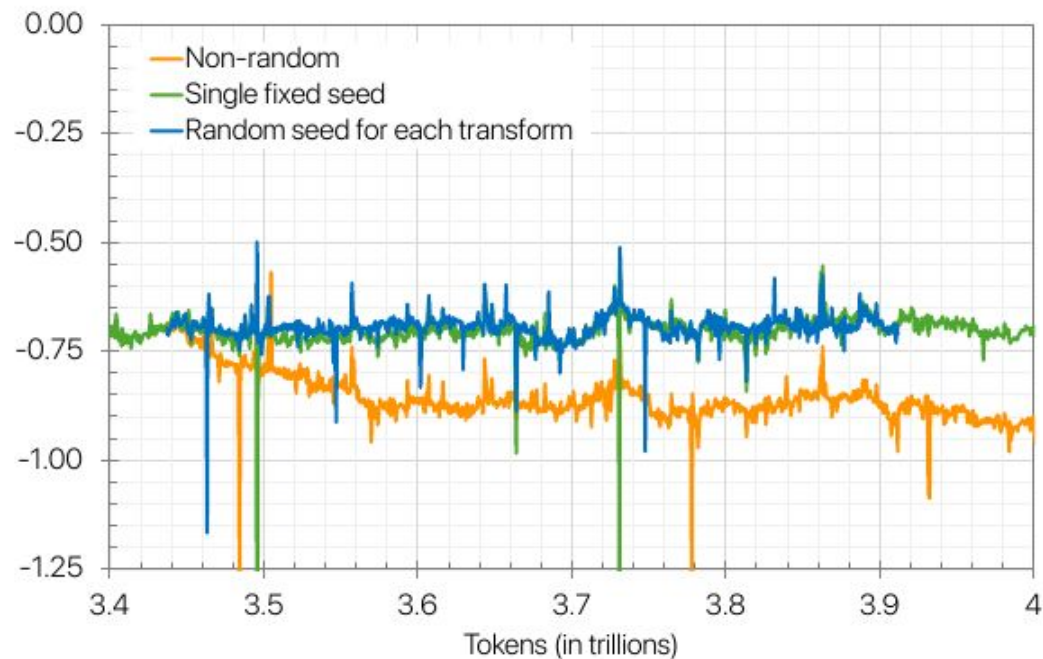
부록 · 16×16과 single fixed seed가 비용 대비 균형점입니다

pplication can negatively affect training when used on certain



Matrix size

4×4는 loss 악화 · 128×128은 소폭 이득 · 16×16 선택



Randomization

random sign 없음은 악화 · 매번 새 seed 이득 없음